📘 전체 코드 흐름 정리 (참고용 — 혼자 복습할 때 읽기)

> 이 문서는 "각 파일이 왜 있는지 / 한 번의 요청이 어떻게 흐르는지 / 꼭 알아야 할 코드 줄"을 정리한 거야. 외울 필요 없고, 헷갈릴 때마다 돌아와서 보면 돼.

---

0. 한 문장 요약

"강사가 아이 정보·수업 기록을 컴퓨터에 넣고(POST), 필요할 때 꺼내 보고(GET), 나중엔 AI가 피드백까지 써주는 웹 서버"

- **넣기 = POST**
- **꺼내기 = GET**
- **고치기 = PATCH**

이게 전부야. 나머지 코드는 전부 "이 3개를 안전하게 처리하기 위한 장치"일 뿐.

---

1. 파일별 역할 (식당에 비유)

웹 서버를 식당이라고 생각하면 각 파일이 직원이야.

파일	식당 비유	하는 일
`main.py`	**식당 입구 / 안내데스크**	앱을 켜고, 손님 요청을 어느 코너로 보낼지 연결
`routers/*.py`	**주방 코너** (메뉴별)	실제 요청을 받아 처리 (넣기/꺼내기 로직)
`models.py`	**창고 선반 설계도**	DB 테이블이 어떻게 생겼는지 (어떤 칸이 있는지)
`schemas.py`	**주문서 양식**	손님이 보낼/받을 데이터 모양 (검사용)
`database.py`	**창고 문 여닫기 담당**	DB 연결 + 들어갔다 나오기

핵심: **파일이 많은 건 일이 복잡해서가 아니라, "역할별로 나눠 놓으면 관리가 쉬워서"** 야. 전부 한 파일에 넣으면 수백 줄짜리 괴물이 돼.

---

2. 한 번의 요청이 흐르는 순서 ⭐ (이게 제일 중요)

예시: 강사가 **"민준이(3번) 최근 수업기록 5개 보여줘"** 라고 요청 (= `GET /lesson/3/lesson-records`)

```
[브라우저/Swagger]
      │  GET /lesson/3/lesson-records
      ▼
① main.py        → "lesson으로 시작하네? → lesson_records_router로 보내자"
      ▼
② lesson_records_router.py → 해당 GET 함수 실행
      │   - child_id = 3 (URL의 {child_id} 자리에서 꺼냄)
      │   - get_db()로 창고 문 열기
```

```
      ▼
③ database.py    → DB 연결 열어줌 (db)
      ▼
④ models.py      → "lesson_records 테이블에서 child_id=3 인 행 찾아" (LessonRecord 사용)
      ▼
   결과(기록 5개) ──► schemas.py(LessonRecordResponse 모양으로 정리) ──► 손님에게 JSON 응답
```

👉 즉 **main → router → (database 열기) → models로 DB 조회 → schemas 모양으로 응답**. 한 요청은 항상 이 길을 지나간다. 이 길만 머리에 있으면 코드가 안 무서워져.

---

3. 파일별 "꼭 알아야 할 코드 줄"

📄 database.py — 창고 문 담당

```python
engine = create_engine(DATABASE_URL)          # DB랑 연결하는 엔진
SessionLocal = sessionmaker(bind=engine)      # 요청마다 새 "연결 세션" 찍어내는 틀
Base = declarative_base()                     # 모든 모델(테이블)의 부모. models.py가 이걸 상속함

def get_db():
    db = SessionLocal()      # 문 열기
    try:
        yield db             # 라우터한테 빌려줌
    finally:
        db.close()           # 끝나면 문 닫기 (중요! 안 닫으면 연결 새어나감)
```

알 것: `get_db`는 "요청 1개당 DB 연결 1개 빌려주고, 끝나면 닫는다". 라우터에서 `Depends(get_db)`로 빌림.

📄 models.py — 테이블 설계도 (창고 선반)

```python
class Child(Base):                    # children 테이블
    id = Column(Integer, primary_key=True)   # 각 아이의 고유 번호 (자동 1,2,3...)
    name = Column(String(100), nullable=False)  # 이름은 필수
    personality = Column(Text)        # 성격 (나중에 AI 피드백용)
    lesson_records = relationship("LessonRecord", back_populates="child")
    #  ↑ "이 아이의 수업기록 목록" (코드에서 child.lesson_records 로 접근 가능)

class LessonRecord(Base):             # lesson_records 테이블
    id = Column(Integer, primary_key=True)
    child_id = Column(Integer, ForeignKey("children.id"))  # ⭐ 어느 아이 기록인지 연결고리
    lesson_date = Column(Date, nullable=False)
    created_at = Column(DateTime, server_default=func.now())  # 저장 시각 자동 입력
    child = relationship("Child", back_populates="lesson_records")
```

알 것:

- `Column(...)` = 테이블의 칸(컬럼) 하나하나.
- `ForeignKey("children.id")` = **"이 수업기록은 children 테이블의 어느 id 소속이냐"** 를 연결하는 끈. 이게 있어야 "민준이 기록만" 골라낼 수 있음.
- `relationship` = 두 테이블을 코드에서 편하게 오가게 해주는 다리. **양쪽에 다 적어야** 에러 안 남.
- `server_default=func.now()` = created_at은 DB가 알아서 시각 넣어줌 → 그래서 우리가 안 보냄.

---

schemas.py — 주문서/영수증 양식 (데이터 검사)

```
class LessonRecordCreate(BaseModel):    # 손님이 POST로 "보내는" 데이터 양식
    child_id: int                       # 필수
    lesson_date: date                   # 필수
    activity: Optional[str] = None      # 있어도 되고 없어도 됨

class LessonRecordResponse(LessonRecordCreate):  # 서버가 "돌려주는" 데이터 양식
    id: int                             # 응답엔 id도 포함
    created_at: Optional[datetime] = None
    class Config:
        orm_mode = True                 # DB 객체를 JSON으로 자동 변환 허용
```

알 것:

- **models vs schemas 차이** (자주 헷갈림!):
  - `models.py` = **DB에 저장되는 실제 모양** (창고 선반)
  - `schemas.py` = **주고받는 데이터의 모양 + 검사** (주문서). 잘못된 데이터 들어오면 여기서 걸러줌.
- `Create` = 받을 때 / `Response` = 줄 때. 보통 Response엔 `id`, `created_at`이 추가됨.

---

routers/lesson_records_router.py — 실제 일하는 곳 ⭐

```
router = APIRouter(prefix="/lesson", tags=["lesson"])   # 이 파일 주소는 전부
/lesson 으로 시작

# [넣기] POST
@router.post("/", response_model=LessonRecordResponse)
def create_LessonRecord(record: LessonRecordCreate, db: Session =
Depends(get_db)):
    new = LessonRecord(child_id=record.child_id, ...)   # 주문서(record) → 창고선반
객체(LessonRecord)
    db.add(new)         # 담고
    db.commit()         # 저장 확정
    db.refresh(new)     # 방금 저장된 최신 상태(자동 생성된 id 등) 다시 읽기
    return new

# [꺼내기] GET — 특정 아이 최근 5개
@router.get("/{child_id}/lesson-records")
def get_lesson_records(child_id: int, db: Session = Depends(get_db)):
```

```
    records = (
        db.query(LessonRecord)                              # lesson_records 테이블에서
          .filter(LessonRecord.child_id == child_id)        # 이 아이 것만 (왼쪽=컬럼, 오
른쪽=들어온 값)
          .order_by(desc(LessonRecord.lesson_date))         # 최신순
          .limit(5)                                         # 5개까지
          .all()                                            # 리스트로 꺼내기
    )
    return records
```

알 것 (DB 조회 4종 세트):

코드	뜻
`db.query(LessonRecord)`	어느 테이블에서
`.filter(컬럼 == 값)`	조건으로 거르기
`.order_by(desc(컬럼))`	정렬 (desc=내림차순=최신먼저)
`.limit(n)`	개수 제한
`.all()` / `.first()`	여러 개 / 딱 하나 꺼내기

넣기(POST) 3종 세트: `db.add()` → `db.commit()` → `db.refresh()`

---

main.py — 입구/연결 담당

```
from routers.children_router import router as children_router
from routers.lesson_records_router import router as lesson_records_router

Base.metadata.create_all(bind=engine)   # models.py에 그린 테이블들을 실제 DB에 생성

app = FastAPI()

app.include_router(children_router)          # /children 요청은 children_router로
app.include_router(lesson_records_router)    # /lesson 요청은 lesson_records_router
로
```

알 것: 라우터를 만들기만 하면 안 되고, **main.py에 ① import + ② include_router 2줄로 "연결"** 해야 작동함. `create_all`은 "models에 설계한 테이블이 DB에 없으면 만들어줘".

---

4. GET 두 종류 (자주 헷갈림)

	`GET /children`	`GET /children/{id}`
뜻	전체 명단	특정 한 명
코드 끝	`.all()`	`.first()`

	`GET /children`	`GET /children/{id}`
비유	아이들 명단표	민준이 카드 한 장

`{id}`는 **"몇 번?"을 넣는 빈칸**. `/children/3` → 3번 아이.

---

5. plan.md Day별 ↔ 코드 매핑

Day	하는 일	만드는/건드리는 파일
Day 1	환경 세팅 + /health	main.py, database.py
Day 2	아이 프로필 API	models.py(Child), schemas.py(Child*), children_router.py (POST/GET/GET{id}/PATCH)
Day 3	수업 기록 API	models.py(LessonRecord), schemas.py(LessonRecord*), lesson_records_router.py (POST/GET) ← **지금 여기**
Day 4	AI 연결	prompt_service.py, llm_service.py (새 파일)
Day 5	피드백 생성 API	feedbacks 테이블 + feedbacks_router.py
Day 6	통합 테스트	(코드보다 데이터 넣고 확인)
Day 7	README 정리	README.md

큰 그림: Day마다 "테이블(models) → 데이터양식(schemas) → API(router) → main에 연결" 이 4단계를 **반복**해. 새 기능이 생길 때마다 이 패턴이 똑같이 돌아감. 이 패턴 하나만 익히면 Day 5, 6도 똑같아.

---

6. 요청 흐름 다시 한 번 (외울 것은 이것뿐)

```
손님 요청 → main.py(어디로 보낼지) → router(처리) → database(연결) → models(DB조회/저장)
→ schemas(모양정리) → 응답
```

이거 하나면 충분해. 막히면 "지금 이 코드가 이 흐름 중 어디 단계지?" 하고 물어보면 길이 보여.